TP 7 React Native :

API utilisée:

https://jsonplaceholder.typicode.com/todos

Installation des packages nécessaires

```
npm install axios
npx expo install @react-native-async-storage/async-storage
npx expo install expo-sqlite
```

Connexion à une API distante (GET)

API utilisée

https://jsonplaceholder.typicode.com/todos

/services/api.js

```
import axios from "axios";

const API_URL = "https://jsonplaceholder.typicode.com";

// axios
export const fetchTodosAxios = async () => {
 const response = await axios.get(`${API_URL}/todos?_limit=10`);
 return response.data;
};

// fetch
export const fetchTodosFetch = async () => {
 const response = await fetch(`${API_URL}/todos?_limit=10`);

 if (!response.ok) {
   throw new Error("Erreur serveur");
 }

 return response.json();
};
```

/screens/TodoListFetchScreen.js

```
import { FlatList, Text, ActivityIndicator, Button } from "react-native";
import { useEffect, useState, useContext } from "react";
import { fetchTodosFetch } from "../services/api";
import { ThemeContext } from "../context/ThemeContext";

export default function TodoListFetchScreen() {
 const [todos, setTodos] = useState([]);
 const [loading, setLoading] = useState(true);
 const [error, setError] = useState(null);

 const { theme, toggleTheme } = useContext(ThemeContext);

 useEffect(() => {
   fetchTodosFetch()
     .then(setTodos)
     .catch(() => setError("Impossible de charger les tâches"))
     .finally(() => setLoading(false));
 }, []);

 if (loading) return <ActivityIndicator size="large" />;

 return (
   <>
     <Button
       title={`Passer en mode ${theme === "light" ? "dark" : "light"}`}
       onPress={toggleTheme}
     />

     {error && <Text>{error}</Text>}

     <FlatList
       data={todos}
       keyExtractor={(item) => item.id.toString()}
       renderItem={({ item }) => (
         <Text
           style={{
             padding: 10,
             color: theme === "dark" ? "#ffffff" : "#000000",
           }}
         >
           {item.title}
         </Text>
       )}
     />
```

```
    </>
  );
}
```

AsyncStorage – Thème persistant

/context/ThemeContext.js

```
import { createContext, useEffect, useState } from "react";
import AsyncStorage from "@react-native-async-storage/async-storage";

export const ThemeContext = createContext();

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState("light");

  useEffect(() => {
    AsyncStorage.getItem("theme").then((value) => {
      if (value) setTheme(value);
    });
  }, []);

  const toggleTheme = async () => {
    const newTheme = theme === "light" ? "dark" : "light";
    setTheme(newTheme);
    await AsyncStorage.setItem("theme", newTheme);
  };

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}
```

App.js

```
import { useContext } from "react";
import { View, StyleSheet } from "react-native";
import { ThemeProvider, ThemeContext } from "./context/ThemeContext";
import TodoListFetchScreen from "./screens/TodoListFetchScreen";

function MainApp() {
  const { theme } = useContext(ThemeContext);

  return (
    <View
      style={[
```

```
        styles.container,
        theme === "dark" ? styles.dark : styles.light,
      ]}
    >
      <TodoListFetchScreen />
    </View>
  );
}

export default function App() {
  return (
    <ThemeProvider>
      <MainApp />
    </ThemeProvider>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    paddingTop: 40,
  },
  light: {
    backgroundColor: "#ffffff",
  },
  dark: {
    backgroundColor: "#121212",
  },
});
```

Lancez l'application pour vérifier que les tâches sont correctement chargées depuis l'API. Ensuite, modifiez volontairement l'URL de l'API et ajoutez un délai artificiel à l'appel afin d'observer l'affichage du loader et du message d'erreur.

SQLite – Mode hors ligne

/services/database.js

```
import * as SQLite from "expo-sqlite";

export const db = SQLite.openDatabaseSync("todos.db");

export const initDB = () => {
  db.execSync(`
    CREATE TABLE IF NOT EXISTS todos (
      id INTEGER PRIMARY KEY,
```

```
    title TEXT
  );
 `);
};

export const addTodoOffline = (title) => {
 db.runSync(
   "INSERT INTO todos (id, title) VALUES (?, ?)",
   [Date.now(), title]
 );
};

export const updateTodoOffline = (id, title) => {
 db.runSync(
   "UPDATE todos SET title = ? WHERE id = ?",
   [title, id]
 );
};

export const loadTodos = () => {
 return db.getAllSync("SELECT * FROM todos");
};
```

/screens/TodoListOfflineScreen.js

```
import { View, Text, FlatList, Button, TextInput } from "react-native";
import { useEffect, useState, useContext } from "react";
import {
 loadTodos,
 addTodoOffline,
 updateTodoOffline,
} from "../services/database";
import { ThemeContext } from "../context/ThemeContext";

export default function TodoListOfflineScreen() {
 const [todos, setTodos] = useState([]);
 const [title, setTitle] = useState("");
 const [editingId, setEditingId] = useState(null);

 const { theme, toggleTheme } = useContext(ThemeContext);

 const refreshTodos = () => {
   setTodos(loadTodos());
 };
```

```
const handleAddOrUpdate = () => {
  if (!title.trim()) return;

  if (editingId) {
    // UPDATE OFFLINE
    updateTodoOffline(editingId, title);
    setEditingId(null);
  } else {
    // ADD OFFLINE
    addTodoOffline(title);
  }

  setTitle("");
  refreshTodos();
};

useEffect(() => {
  refreshTodos();
}, []);

return (
  <>
    {/* Theme toggle */}
    <Button
      title={`Passer en mode ${theme === "light" ? "dark" : "light"}`}
      onPress={toggleTheme}
    />

    {/* Add / Update */}
    <View style={{ padding: 10 }}>
      <TextInput
        placeholder="Tâche offline"
        value={title}
        onChangeText={setTitle}
        style={{
          borderWidth: 1,
          padding: 10,
          marginBottom: 10,
        }}
      />

      <Button
        title={editingId ? "✏️ Mettre à jour" : "➕ Ajouter hors ligne"}
        onPress={handleAddOrUpdate}
      />
    </View>
```

```
    {todos.length === 0 ? (
      <Text style={{ textAlign: "center", marginTop: 20 }}>
        Aucune tâche disponible hors ligne
      </Text>
    ) : (
      <FlatList
        data={todos}
        keyExtractor={(item) => item.id.toString()}
        renderItem={({ item }) => (
          <View
            style={{
              flexDirection: "row",
              justifyContent: "space-between",
              padding: 10,
            }}
          >
            <Text>{item.title}</Text>

            <Button
              title="✏️"
              onPress={() => {
                setTitle(item.title);
                setEditingId(item.id);
              }}
            />
          </View>
        )}
      />
    )}
  </>
);
}
```

Modifier App.js pour l'affichage offline

App.js

```
import { useEffect, useState, useContext } from "react";
import { View, StyleSheet, ActivityIndicator } from "react-native";
import { initDB } from "./services/database";
import { ThemeProvider, ThemeContext } from "./context/ThemeContext";
import TodoListOfflineScreen from "./screens/TodoListOfflineScreen";

function MainApp() {
```

```
 const { theme } = useContext(ThemeContext);

 return (
   <View
     style={[
       styles.container,
       theme === "dark" ? styles.dark : styles.light,
     ]}
   >
     <TodoListOfflineScreen />
   </View>
 );
}

export default function App() {
 const [dbReady, setDbReady] = useState(false);

 useEffect(() => {
   const prepareDb = async () => {
     await initDB();   // attendre SQLite
     setDbReady(true); // OK pour afficher l'app
   };

   prepareDb();
 }, []);

 if (!dbReady) {
   return <ActivityIndicator size="large" />;
 }

 return (
   <ThemeProvider>
     <MainApp />
   </ThemeProvider>
 );
}


const styles = StyleSheet.create({
 container: {
   flex: 1,
   paddingTop: 40,
 },
 light: {
   backgroundColor: "#ffffff",
 },
```

```
  dark: {
    backgroundColor: "#121212",
  },
});
```

Exercice supplémentaire :

Supprimer une tâche hors ligne (SQLite)

À faire :

- Ajouter un bouton 🗑️ à côté de chaque tâche
- Supprimer la tâche de SQLite
- Rafraîchir la liste